Wissensintegration in KI-Tutorensystemen: Der Einfluss von Retrieval-Augmented-Generation auf die Antwortqualität von KI-Modellen

Eine technische und prozessorientierte Analyse

Maturitätsarbeit

Verfasser:

Luka Simonovic

Kantonsschule Zürcher Unterland

Schuljahr 2025/2026

Inhaltsverzeichnis

1. Einleitung 4
1.1 Persönliche Motivation und Problemstellung 4
1.2 Fragestellung 4
1.3 Zielsetzung 5
1.4 Methodik und Vorgehensweise 6
1.5 Voraussetzungen zum Verständnis 6
2. Der Arbeitsprozess (Entwicklungsphasen) 7
2.1 März 2025: Die Initialzündung und erste Prototypen 7
2.2 April 2025: Das grosse Refactoring 7
2.3 Mai 2025: RAG-Integration und OCR 7
2.4 Juni 2025: Prüfungssimulation und Feinjustierung 8
2.5 Juli/August 2025: Abschluss und UI-Polishing 8
3. Wie funktioniert Gymiboost? (Theorie und Implementierung) 9
3.1 Technologisches Fundament: Warum Flask? 9
3.2 Das Herzstück: Retrieval-Augmented Generation (RAG) 9
3.3 Vektordatenbank: Lokale Kontrolle mit ChromaDB 10
3.4 Evolution der KI-Modelle: Von DeepSeek zu GPT-5 11
3.5 Backend-Architektur und Datenfluss 11
3.6 Streaming und Frontend-Interaktion 12
3.7 Integration von Bezahlsystemen 12
4. Technische Hürden und Fehleranalyse 13
4.1 Das Datenbank-Dilemma: SQLite vs. MySQL 13
4.2 Das Emoji-Problem (Encoding) 14
4.3 Die Herausforderung der mathematischen Notation (MathJax) 14
4.4 Kontext-Management und Speicherlast 14
4.5 Hardware-Grenzen und Ambitionen 15
5. Resultate und Beantwortung der Fragestellung 16
5.1 Erreichung der Minimalziele 16
5.2 Erreichung der Maximalziele 16
5.3 Funktionsweise des Endprodukts 17
5.4 Antwort auf die Leitfrage 17

6. Fazit und Reflexion 18
6.1 Zusammenfassung 18
6.2 Persönliche Reflexion 18
7. Quellenverzeichnis 19
8. Eigenständigkeitserklärung 22

1. Einleitung

1.1 Persönliche Motivation und Problemstellung

Die Zentrale Aufnahmeprüfung (ZAP) im Kanton Zürich stellt für Tausende von Sechstklässlern eine der ersten grossen Hürden in ihrer schulischen Laufbahn dar. Es ist ein Selektionsinstrument, das weit mehr prüft als nur rohe kognitive Leistungsfähigkeit oder allgemeine Intelligenz. Vielmehr verlangt sie den Abruf von hochspezifischem, kontextabhängigem Wissen und die Anwendung präziser, fast schon ritualisierter Lösungsstrategien, insbesondere in der Geometrie und im Sprachverständnis. In einer Zeit, in der Künstliche Intelligenz (KI) und Large Language Models (LLMs) wie ChatGPT scheinbar jede Frage der Welt beantworten können, wirkt der traditionelle Vorbereitungsprozess – oft geprägt von Stapeln alter Papierprüfungen und teuren Nachhilfekursen – anachronistisch und ineffizient. Doch wer versucht, moderne KI-Tools für diesen spezifischen Zweck einzusetzen, stösst schnell an ernüchternde Grenzen.

Die gängigen Modelle sind Generalisten. Sie beherrschen zwar den Konversationsstil perfekt und verfügen über ein breites Weltwissen, doch sie kennen zum Beispiel die Feinheiten der Zürcher ZAP-Prüfung von 2023 nicht im Detail. Fehlt ihnen dieses spezifische Wissen, neigen sie dazu, Fakten glaubwürdig zu erfinden oder Rechenwege vorzuschlagen, die zwar mathematisch möglich, aber im Schweizer Schulkontext unüblich sind. Im Bildungskontext ist dieses Phänomen, bekannt als "Halluzinieren", ein fataler Fehler, der das Vertrauen der Lernenden untergräbt und den Lernerfolg gefährdet. Das primäre Ziel dieser Arbeit war es daher, eine technologische Brücke zu schlagen.

Eine Brücke zwischen der brachialen generativen Kraft moderner KI-Modelle und den strikten, lokalen Anforderungen des Zürcher Bildungssystems. Das Projekt "Gymiboost" entstand aus dem Wunsch heraus, Technik nicht als blosse Spielerei, sondern als echten Problemlöser einzusetzen. Es ging darum, ein Werkzeug zu schaffen, das dem Nutzer die mühsame Arbeit der Kontextualisierung abnimmt und verlässliche Antworten liefert. Um das Projekt im Rahmen einer Maturitätsarbeit realisierbar und überschaubar zu halten, habe ich mich bei der Entwicklung und dem Training der Datenbasis explizit auf die Anforderungen des Langgymnasiums (Übertritt nach der 6. Primarklasse) fokussiert. Die Strukturen und Aufgabentypen für das Kurzgymnasium unterscheiden sich signifikant, weshalb eine Ausweitung auf beide Stufen den zeitlichen und technischen Rahmen dieses Ein-Personen-Projekts gesprengt hätte.

1.2 Fragestellung

Die leitende Fragestellung dieser Arbeit lautet: Kann ich als einzelner Entwickler eine RAG-basierte Web-Applikation konzipieren und implementieren, die eine halluzinationsfreie, prüfungsspezifische Unterstützung für die ZAP (Langgymnasium) bietet, und lassen sich die dabei auftretenden technischen sowie architektonischen Hürden mit begrenzten Ressourcen überwinden?

1.3 Zielsetzung

Die Ziele dieser Arbeit wurden im Vorfeld klar definiert und in Minimal- sowie Maximalziele unterteilt, um den Entwicklungsprozess besser strukturieren zu können. Diese Zielsetzung dient im späteren Verlauf der Arbeit (siehe Kapitel 5) als Massstab für den Erfolg des Projekts.

Minimalziele:

Die absolute Basis des Projekts sollte ein funktionaler Prototyp sein, der die Machbarkeit des

RAG-Ansatzes (Retrieval-Augmented Generation) im schulischen Kontext beweist.

● Dokumentenverarbeitung: Implementierung einer Pipeline, die Text zuverlässig aus PDFs und Bildern (mittels OCR) extrahiert. Hierbei sollten Python-Bibliotheken wie PyMuPDF und Tesseract zum Einsatz kommen, um auch ältere Prüfungsarchive digital zugänglich zu machen.

● RAG-Engine: Aufbau einer lokalen Vektordatenbank (ChromaDB), die eine semantische Suche über die extrahierten Dokumente ermöglicht, um den KI-Modellen den nötigen Kontext zu liefern.

● Chat-Interface: Eine webbasierte Oberfläche, die Antworten der KI in Echtzeit, token-by-token, streamt und dem Nutzer erlaubt, intuitiv und ohne technische Vorkenntnisse mit der KI zu interagieren.

● Qualitätssicherung: Die Antworten müssen durch den RAG-Kontext nachweislich präziser, relevanter und faktentreuer sein als die eines "nackten", nicht kontextualisierten LLMs.

Maximalziele:

Sollte die Basis stabil stehen, war die Erweiterung zu einer vollwertigen Lernplattform geplant.

● Erweiterte Verarbeitung: Unterstützung komplexer Formate wie handschriftliche Notizen von Schülern und die korrekte Interpretation mathematischer Formeln.

● UI/UX: Ein professionelles Interface mit Markdown-Rendering, Code-Highlighting und einer Verwaltung für verschiedene Chat-Sitzungen (inkl. Chat-History), das modernen Standards entspricht.

● Features: Simulation von echten Prüfungssituationen (Timer, PDF-View), automatische Generierung von neuen, synthetischen Übungsaufgaben basierend auf alten Mustern und eine Exportfunktion für Ergebnisse.

● Architektur: Eine skalierbare und klar übersichtliche Architektur, die mir Erlaubt, zukünftige Nutzer und Features einfach einzubauen.

1.4 Methodik und Vorgehensweise

Die Entwicklung folgte keinem vorgeplanten, akademischen Plan, sondern einem «learning by doing» iterativen Ansatz, der in der Softwareentwicklung oft als "Agile Development" bezeichnet wird.

Der Fokus lag ganz klar auf "Learning by Doing". Da sich viele der verwendeten Technologien, insbesondere Vektordatenbanken, Embedding-Modelle und RAG-Architekturen – rasant entwickeln, waren klassische Lehrbücher, erstens teuer, und zweitens oft nutzlos oder zum Zeitpunkt des Lesens bereits veraltet. Stattdessen dienten aktuelle Online-Communities wie Stackoverflow, GitHub Issues und Reddit als primäre Wissensquellen. Wenn ein Fehler auftrat – und das passierte in der Arbeit mit experimentellen KI-Modellen häufig –, war die kollektive Intelligenz dieser Plattformen mein erster Anlaufpunkt. Zudem wurde KI selbst als Werkzeug genutzt (sogenanntes AI-Assisted Coding). Teile des Codes, insbesondere repetitive HTML-Strukturen, Boilerplate-Code für Flask oder komplexe Regex-Muster für die Textverarbeitung, wurden mithilfe von KI-Modellen generiert und anschliessend von mir überarbeitet, getestet und integriert. Dies spiegelt die moderne Realität der Softwareentwicklung wider: Der Programmierer wird zunehmend zum Architekten, der KI-generierte Bausteine prüft, korrigiert und zu einem funktionierenden Ganzen zusammensetzt. Die Arbeit dokumentiert diesen Prozess ungeschönt, einschliesslich der Sackgassen und Fehlentscheidungen, denn gerade das Scheitern an technischen Hürden lieferte oft die wertvollsten Erkenntnisse für das Endprodukt.

1.5 Voraussetzungen zum Verständnis

Um den technischen Ausführungen in dieser Arbeit folgen zu können, sind grundlegende Kenntnisse in der Informatik hilfreich, aber nicht zwingend erforderlich. Ein Verständnis der folgenden Konzepte erleichtert das Lesen:

- Grundlegende Programmierkenntnisse: Idealerweise in Python, da dies die Sprache des Projekts ist.
- Webarchitektur: Das Verständnis der Trennung von Frontend (was der Nutzer sieht) und Backend (wo die Logik läuft).
- Datenbanken: Eine grobe Vorstellung davon, wie Daten gespeichert und abgerufen werden.

Alle spezifischen Fachbegriffe wie "RAG" (Retrieval-Augmented Generation), "Vektordatenbank", "Embeddings" oder "LLM" (Large Language Model) werden im Verlauf der Arbeit an entsprechender Stelle erklärt.

2. Der Arbeitsprozess (Entwicklungsphasen)

Die Entwicklung von "Gymiboost" verlief nicht linear, sondern in intensiven Schüben, die von technischen Durchbrüchen, Motivations- Ab- und Aufschwüngen sowie Rückschlägen geprägt waren. Dieser Abschnitt zeichnet den zeitlichen Verlauf des Projekts anhand der Entwicklungsphasen nach.

2.1 März 2025: Die Initialzündung und erste Prototypen

Der Startschuss für das Projekt fiel im März. Zu diesem Zeitpunkt war das Ziel noch relativ unklar: Ich wollte die Möglichkeiten moderner KI nutzen, um schulische Probleme zu l sen. Der erste Code war simpel aufgebaut und diente als Machbarkeitsstudie. Ich startete mit einem einfachen Flask-Boilerplate (einem Grundgerüst für Web-Apps in Python) und integrierte einen rudimentären Chatbot. Dieser basierte auf "Ollama", einer Software, die es ermöglicht, KI-Modelle lokal auf dem eigenen Laptop laufen zu lassen, ohne auf teure Cloud-APIs angewiesen zu sein. Alles war zu diesem Zeitpunkt noch sehr roh und "unrefined". Es ging primär darum, ein Gefühl für die Technologie zu bekommen und zu testen, ob meine vorhandene Hardware überhaupt leistungsfähig genug ist, um diese rechenintensiven Modelle in akzeptabler Geschwindigkeit zu betreiben.

2.2 April 2025: Refactoring

Am 4. April kam es zu einem entscheidenden Wendepunkt. Ich realisierte, dass mein erster Entwurf, der mittlerweile aus unübersichtlichem "Spaghetti-Code" bestand, nicht skalierbar war. Die Funktionen waren zu eng miteinander verwoben; eine Änderung an der Datenbank führte oft zu Fehlern im Chat-Interface. Wenn ich komplexe Features wie die gezielte Dokumentensuche (RAG) hinzufügen wollte, musste ich die Architektur von Grund auf sauberer gestalten. Also entschied ich mich schweren Herzens, den bisherigen Code weitgehend zu verwerfen und das Fundament neu zu schreiben. Ich strukturierte die App modular, trennte die Datenbanklogik strikt vom Frontend und bereitete den Code so systematisch wie möglich darauf vor, das RAG-System zu unterstützen. Dies war für mich ein Prägender Moment in diesem Projekt.

2.3 Mai 2025: RAG-Integration und OCR

Der Mai war der Monat der grossen technischen Implementierungen. Das RAG-System wurde nun voll integriert, und ich baute ChromaDB als lokale Vektordatenbank ein. Dies ermöglichte es der KI erstmals, "in Dokumenten zu lesen". Ein wichtiger Meilenstein war der 21. Mai, als ich die OCR-Funktion (Optical Character Recognition) hinzufügte. Das war entscheidend, da viele alte ZAP-Prüfungen nur als gescannte Bilder (PDFs ohne Textlayer) vorliegen. Ohne eine integrierte OCR-Pipeline wäre die App für einen Grossteil der relevanten Dokumente blind gewesen. Ende Mai war die App schon eine art «Proof of concept», welche gewisses potenzial zeigte.

Ich verbaute auch eine erste, experimentelle Integration des Bezahldienstleisters Stripe, um theoretisch zukünftige Abonnements verwalten zu können.

2.4 Juni 2025: Prüfungssimulation und Feinjustierung

Im Juni verschob sich der Fokus von der reinen Technik hin zu den pädagogischen Features und der UX (User Experience bzw. Nutzererlebnis). Ich implementierte Testsimulationen, die es Schülern erlauben, unter Zeitdruck zu üben, sowie die automatische Generierung von Aufgaben. Besonders die Darstellung mathematischer Inhalte im Browser stellte mich vor grosse Herausforderungen.

Man sieht in meinen Commit-Nachrichten aus dieser Zeit den wachsenden Frust über technische Details: "MATHJAX DISPLAYS CORRECTLY ON TASK GENERATION NOW FFS". Dieser Eintrag markiert den Moment, als es mir endlich gelang, komplexe Brüche und Wurzeln mittels der Bibliothek MathJax dynamisch und korrekt im Chatfenster rendern zu lassen – ein Feature, das für eine Mathe-Lernplattform unverzichtbar ist, und welches in dieser Arbeit noch mehrmals erwähnt wird.

2.5 Juli/August 2025: Abschluss und UI-Polishing

In den letzten beiden Monaten konzentrierte ich mich auf den Feinschliff und die Stabilität. Das Design (UI) wurde komplett überarbeitet, um moderner und zugänglicher zu wirken. Ich fügte Funktionen wie den PDF-Export für generierte Übungsblätter hinzu, damit Schüler auch offline lernen können, oder Nachhilfe-Lehrer Arbeitsblätter in Papierform an ihre Schüler verteilen können. Daraufhin integrierte ich noch zwei Features, die Kurs-Funktion, die es Nutzern erlaubt, interaktive, KI generierte Kurse zu Themen ihrer Wahl fertigen zu lassen und zu absolvieren, und einen Fortschritts-Tracker, der die Aktivität der Nutzer auf der Plattform verfolgt, um abzuschätzen, wie sie mit der Vorbereitung vorankommen. Optimierungen zum Front-End und der Speicherverwaltung rundeten die Entwicklung ab, bevor ich mich an die Dokumentation und das Schreiben dieser Arbeit machte.

3. Wie funktioniert Gymiboost? (Theorie und Implementierung)

Bevor auf die spezifische Implementierung eingegangen wird, ist es notwendig, die technologischen Entscheidungen und die theoretischen Konzepte hinter der App zu beleuchten. Warum wurden bestimmte Werkzeuge gewählt und wie greifen sie ineinander?

3.1 Technologisches Fundament: Warum Flask?

Im Python-Ökosystem gibt es zwei Platzhirsche für Webentwicklung: Django und Flask. In jüngerer Zeit hat sich zudem FastAPI als starker, moderner Konkurrent gezeigt. Die Wahl des Frameworks bestimmt einen grossen Teil der Architektur und Flexibilität der gesamten Anwendung. Django folgt dem Prinzip "Batteries Included". Es liefert alles "ab Werk" mit: ein mächtiges ORM (Object-Relational Mapping), ein fertiges Authentifizierungssystem, ein Admin-Panel und eine strikte Ordnerstruktur. Für viele Entwickler ist das ein Segen, da es Standardprobleme sofort löst. Doch für dieses Projekt fühlte sich Django zu schwerfällig und bevormundend an. Ausserdem hatte ich mit Django kaum erfahrung, was den Entwicklungsprozess weiter in die Länge ziehen würde.

Wenn man, wie ich bei diesem Lernprojekt, im Detail verstehen will, wie genau die Daten von der Datenbank ins Frontend gelangen und wie der KI-Datenstrom verarbeitet wird, versteckt Django zu viel "Magie" hinter seinen Abstraktionen.

Flask hingegen ist ein Mikro-Framework. Der Flask-spezifische alltägliche Syntax lässt sich im Grunde gesehen auf eine A4-Seite reduzieren. Es liefert im Kern nur zwei Dinge: Werkzeug (für die Server-Kommunikation) und Jinja2 (für Templates). Alles andere – Datenbankanbindung, Login-Logik, Formulare – muss der Entwickler selbst entscheiden und einbauen. Das klingt nach Mehrarbeit, und das ist es auch. Aber es bietet maximale Freiheit. Flask lässt mir als Entwickler sehr viel offen. Ich kann die Architektur genau so bauen, wie ich sie brauche, ohne gegen die Konventionen des Frameworks kämpfen zu müssen. Wie Diskussionen in Entwicklerforen bestätigen, ist Flask ideal für kleine bis mittelgrosse Projekte, bei denen man die volle Kontrolle behalten möchte und nicht unnötigen Ballast mitschleppen will. Die Entscheidung fiel also auf Flask, nicht weil es objektiv "besser" ist, sondern weil es meinem Entwicklungsstil besser entsprach.

3.2 Das Herzstück: Retrieval-Augmented Generation (RAG)

Künstliche Intelligenz, speziell LLMs wie GPT-4, sind beeindruckend wortgewandt und kreativ. Aber sie haben kein echtes Wissen über die Welt nach ihrem Trainingsschluss ("Cut-off Date"), und sie haben keinen Zugriff auf Fachwissen zu Nischenthemen lokale, private Dateien. Wenn man ein LLM nach der Zürcher ZAP-Mathematikprüfung 2024 fragt, wird es entweder zugeben, dass es diese nicht kennt, oder, schlimmer, eine plausibel klingende, aber falsche Prüfung erfinden.

RAG ist die technologische Antwort auf dieses Problem. Es kombiniert die Sprachfähigkeit

des LLMs mit einer externen, vertrauenswürdigen Wissensdatenbank. Der Prozess, den ich in rag_utils.py implementiert habe, lässt sich vereinfacht so darstellen:

1. Wissensspeicher (Ingestion): Man nimmt die relevanten Dokumente (PDFs der alten Prüfungen und Lösungen) und zerlegt sie in kleine, verdauliche Textabschnitte, sogenannte "Chunks".

2. Vektorisierung (Embeddings): Diese Chunks werden durch ein Embedding-Modell in Vektoren umgewandelt. Ein Vektor ist eine lange Liste von Zahlen, die die semantische Bedeutung des Textes repräsentieren. Sätze, die inhaltlich ähnlich sind, haben mathematisch ähnliche Vektoren (geringe Distanz im Vektorraum), auch wenn sie unterschiedliche Wörter verwenden, sind sie wortwörtlich «Näher bei einander» als andere.

3. Abfrage (Query): Wenn der Nutzer eine Frage stellt (z.B. "Wie löse ich die Geometrieaufgabe mit dem Dreieck?"), wird auch diese Frage vektorisiert.

4. Retrieval: Die Datenbank sucht nun blitzschnell die Text-Chunks, deren Vektoren der Frage am ähnlichsten sind. Dabei habe ich einen "Hybrid Search"-Ansatz implementiert: Die Funktion enhance_query_with_year_and_subject versucht zusätzlich, Metadaten wie "2022" oder "Geometrie" aus der Frage zu extrahieren, um die Suche vorzufiltern und die Präzision zu erhöhen.

5. Generierung: Das LLM erhält die ursprüngliche Frage und die gefundenen Textstellen (den Kontext) mit der strikten System-Anweisung: "Beantworte die Frage des Nutzers nur basierend auf diesen beigefügten Informationen." Dadurch wird das LLM vom unzuverlässigen "Wisser" zum präzisen "Verarbeiter". Es muss nicht mehr Fakten auswendig wissen, sondern nur noch den bereitgestellten Text analysieren und zusammenfassen.

3.3 Vektordatenbank: Lokale Kontrolle mit ChromaDB

Für die Speicherung und Durchsuchung der Vektoren brauchte es eine spezialisierte Datenbank. Der Industriestandard ist oft Pinecone, eine reine Cloud-Lösung, die extrem skalierbar und schnell ist. Doch Pinecone hat Nachteile: Es kostet Geld, sobald man über den kostenlosen Bereich hinauswächst, und die sensiblen Prüfungsdaten liegen auf fremden Servern. Für ein Schülerprojekt, bei dem das Budget bei null liegt und der Datenschutz eine Rolle spielt, war das keine Option. Die Wahl fiel auf ChromaDB.

Chroma ist Open-Source und läuft lokal als Teil der Python-Applikation. Es speichert die Daten einfach in Dateien auf der Festplatte meines Servers. Das bedeutet: Keine Cloud-Kosten, keine Netzwerklatenz zur Datenbank und volle Kontrolle über die Daten. Zwar skaliert Chroma nicht so gut wie Pinecone, wenn man Millionen von Dokumenten hat, aber für die Menge an ZAP-Prüfungen (einige Dutzend mittelgrosser PDFs) ist es mehr als ausreichend leistungsfähig und bietet eine sehr einfache, Python-freundliche API.

3.4 Evolution der KI-Modelle: Von DeepSeek zu GPT-5

Ein zentraler Aspekt des Projekts war die dynamische Anpassung der verwendeten KI-Modelle. Da sich die Landschaft der Large Language Models (LLMs) während der Entwicklungsphase rasant veränderte, passte auch ich meine Strategie kontinuierlich an, um das beste Verhältnis aus Leistung und Kosten zu erzielen. Diese Evolution lässt sich in drei Phasen unterteilen:

● Phase 1: Der Start mit DeepSeek (V3 & R1): Zu Beginn des Projekts setzte ich auf DeepSeek V3 und das spezialisierte Reasoning-Modell DeepSeek R1. Diese Modelle boten zum damaligen Zeitpunkt eine beeindruckende Qualität bei extrem niedrigen Kosten. Besonders das R1-Modell war entscheidend, um mathematische Lösungswege logisch korrekt herzuleiten, anstatt nur Ergebnisse zu raten.

● Phase 2: Skalierung mit 3.1 und GPT-4.1: Mit zunehmender Komplexität der Aufgaben, insbesondere bei anspruchsvollen Textanalysen, stiess ich an Grenzen. Ich integrierte zwischenzeitlich neuere Versionen (wie DeepSeek 3.1) und für besonders schwierige Reasoning-Tasks das Modell GPT-4.1. Dies war eine Übergangsphase, in der ich lernte, Aufgaben je nach Schwierigkeitsgrad an verschiedene Modelle zu delegieren.

● Phase 3: Finaler Ansatz mit dem GPT-5 Ökosystem: Der "Gamechanger" für die finale Version von Gymiboost war die Veröffentlichung von GPT-5. Dieses Modell bot eine Top-Performance, die sowohl die älteren DeepSeek-Modelle als auch Konkurrenten wie Claude in den Schatten stellte – und das bei einer überraschend günstigen Kostenstruktur.

In der finalen Architektur von Gymiboost kommen nun spezialisierte Varianten zum Einsatz:

○ GPT-5 (Standard): Dient als "Main Agent". Es führt den Chat, übernimmt die komplexen Prüfungssimulationen und strukturiert ganze Lernkurse im Hintergrund.

○ GPT-5-mini: Dieses kleinere, schnellere Modell ist das "Arbeitspferd" für repetitive Aufgaben. Es generiert synthetische Übungsaufgaben, schreibt kleinere Lernmodule und übernimmt Aufgaben, bei denen Geschwindigkeit wichtiger ist als tiefe philosophische Reasoning-Fähigkeit. Diese hybride Strategie ermöglicht es Gymiboost, die Maximale Leistung aus der KI zu würgen, ohne die Kosten eines Enterprise-Projekts zu verursachen.

3.5 Backend-Architektur und Datenfluss

Der Einstiegspunkt der Anwendung ist die Datei app.py. Hier laufen alle Fäden zusammen. Das Backend übernimmt mehrere Aufgaben gleichzeitig: Es ist Webserver (liefert HTML an den Browser), API-Server (verarbeitet JSON-Daten vom Frontend) und Controller (steuert die Logik zwischen Datenbank und KI). Ich verwende SQLAlchemy als ORM (Object-Relational Mapper), was bedeutet, dass Datenbanktabellen nicht mit rohem SQL erstellt werden müssen, sondern als Python-Klassen definiert sind (User, ChatSession,

Message). Das macht den Code, wie in files config.py und models.py zu sehen ist, deutlich lesbarer, wartbarer und schützt automatisch vor vielen Sicherheitslücken wie SQL-Injection-Angriffen. Ein interessantes Detail ist die Trennung der Konfiguration im file config.py. Hier unterscheide ich zwischen DevelopmentConfig und ProductionConfig. Das erlaubt es mir, lokal auf meinem Laptop mit einer einfachen SQLite-Datenbank zu entwickeln, aber auf dem Server eine leistungsstärkere MySQL-Datenbank zu nutzen – eine Entscheidung, die später noch für unerwartete Probleme sorgen sollte (siehe Kapitel 4).

3.6 Streaming und Frontend-Interaktion

Ein modernes Chat-Interface muss sich lebendig und reaktionsschnell anfühlen. Niemand will 10 Sekunden auf einen blinkenden Cursor starren, bis plötzlich der ganze Textblock erscheint.

Die Lösung ist Streaming. In Flask ist das technisch nicht trivial, da Python standardmässig synchron arbeitet. Die Funktion stream_and_save_response nutzt Python-Generatoren (yield), um die Antwort des LLMs Stück für Stück (Token für Token) an den Browser zu senden, sobald sie generiert werden. Im Frontend (chat.js) muss dieser Datenstrom empfangen und sofort dargestellt werden.

Das allein wäre einfach, aber die Anforderung, Markdown (für Textformatierung) und LaTeX (für mathematische Formeln) in Echtzeit zu rendern, machte es kompliziert. Wenn ein halber LaTeX-Befehl wie \frac{1 ankommt, darf der Renderer nicht abstürzen, sondern muss warten, bis der Rest }{2} da ist. Das Zusammenspiel zwischen dem Streaming-Backend und dem MathJax-Rendering im Frontend war eine der grössten technischen Herausforderungen des Projekts.

3.7 Integration von Bezahlsystemen

Auch wenn Gymiboost primär ein Schulprojekt ist, wurde die Monetarisierung theoretisch mitgedacht. Die Integration von Stripe zeigt den Versuch, eine professionelle SaaS-Struktur (Software as a Service) aufzubauen. Nutzer können Abonnements abschliessen, und der Zugriff auf bestimmte Premium-Bereiche der App (wie die Prüfungssimulation oder unbegrenzte Aufgabengenerierung) wird über Python-Decorators (@subscription_required) gesteuert. Das zeigt, wie tief die Geschäftslogik in den Code verwoben ist: Ein nicht zahlender Nutzer wird vom Server rigoros auf die Bezahlseite umgeleitet, noch bevor die Seite geladen wird.

4. Technische Hürden und Fehleranalyse

In diesem Kapitel soll möglichst ehrlich aufgezeigt werden, wo ich mir selbst im Weg stand. Es ist einfach, über Erfolge zu schreiben, aber man, oder zumindest ich, lerne am meisten aus den Fehlern und den Hürden der Entwicklung. Die Entwicklung war geprägt von Momenten, in denen Theorie und Praxis kollidierten, und dadurch für extrem viel Frust Kopfschmerzen sorgten.

4.1 Das Datenbank-Dilemma: SQLite vs. MySQL

Zu Beginn nutzte ich SQLite. Es ist einfach, benötigt keine Server-Installation ("Serverless") und ist direkt in Python integriert. Alles funktionierte lokal wunderbar. Doch als ich die App produktiver machen wollte und auf eine MySQL-Datenbank wechselte, begannen die komplett unerklärbaren Abstürze.

Das Problem war subtil und lehrte mich viel über Datentypen, von denen ich vorher eigentlich nur im Vorbeigehen erfahren habe: Ich speicherte die kompletten Chatverläufe und den gefundenen RAG-Kontext in der Datenbank. In SQLite ist ein Textfeld praktisch unbegrenzt gross. In MySQL hat der Standard-Datentyp TEXT jedoch ein hartes Limit von 64 Kilobyte. Ein langer Chat mit viel Kontext sprengt dieses Limit sofort. Ich habe mir hier selbst ins Bein geschossen, indem ich die Unterschiede zwischen Entwicklungs- und Produktionsumgebung ignorierte. Die Fehlersuche dauerte mehrere Stunden über fast 3 Tage, bis ich verstand, dass Daten einfach "abgeschnitten" wurden, was zu korrupten JSON-Strukturen führte. Die Lösung war die Migration des Datentyps zu LONGTEXT (bis zu 4 GB). Eine triviale Änderung im Code, aber eine fundamentale und schlicht essenzielle Lektion in Datenbank-Theorie.

4.2 Das Emoji-Problem (Encoding)

Ein weiteres Detail, das mich fast zur Verzweiflung brachte, waren Emojis. Die KI nutzt gerne Emojis wie ✅ oder ❌, um Feedback zu geben oder Listen aufzulockern. Als ich diese Antworten in die MySQL-Datenbank speichern wollte, stürzte die Anwendung ab oder zeigte nur Fragezeichen an.

Der Grund dafür ist: MySQL nutzt standardmässig oft ein Encoding (utf8), das aus historischen Gründen nur 3 Bytes pro Zeichen speichert. Emojis benötigen aber 4 Bytes. Ich musste die gesamte Datenbankkonfiguration und die Verbindungsparameter explizit auf utf8mb4 umstellen.

Das war ein klassischer Fall von "es funktioniert auf meinem Laptop aber nicht auf dem Server".

4.3 Die Herausforderung der mathematischen Notation (MathJax)

Mathematik im Web darzustellen, ist schwer. LaTeX ist der Standard für wissenschaftliche Formeln, aber Webbrowser verstehen nativ kein LaTeX. Bibliotheken wie MathJax übersetzen es in lesbare Formeln. Das Problem war die Schnittstelle zwischen der "kreativen" KI und dem strikten MathJax-Renderer.

Die KI variiert ihren Output gerne. Mal liefert sie eine Formel als x^2, mal als x^2, mal als \(x^2\). MathJax erwartet aber ein ganz bestimmtes Format, um zu rendern. Wenn die KI variiert, bleibt das Frontend kaputt und zeigt kryptischen Code statt schöner Brüche an.

Ich habe zunächst versucht, das mit Python-Code zu fixen (komplexe Regex-Ersetzungen), was aber scheiterte, da die Variationen zu gross waren. Die Lösung war schliesslich nicht technischer Natur, sondern linguistischer: Prompt Engineering. Im System-Prompt in app.py musste ich die KI fast schon anschreien: "Verwende IMMER LaTeX-Syntax für mathematische Ausdrücke... Verwende niemals normale Klammern ( ) für Mathematik - immer \(\) verwenden". Ich lernte, dass Höflichkeit gegenüber der KI ("Bitte erstelle...") zu schlechteren Ergebnissen führt als strikte, militärische Befehle und klare Constraints.

4.4 Kontext-Management und Speicherlast

Ein Design-Fehler, den ich früh beging und der bis zum Ende blieb, betrifft das "Gedächtnis" des Bots. Ich speicherte den gesamten gefundenen RAG-Kontext (die extrahierten Buchseiten) direkt im ChatSession-Objekt in der Datenbank. Das ist extrem ineffizient. Es bläht die Datenbank unnötig auf. Jedes Mal, wenn ein Nutzer den Chat öffnet, müssen Megabytes an Text geladen werden(viel), was die Ladezeiten verlängert. Eine sauberere Lösung wäre gewesen, den Kontext nur temporär im RAM zu nutzen oder dynamisch bei Bedarf neu zu laden. Hier habe ich halb-bewusst «Technical Debt» aufgebaut. Es funktioniert, ist aber architektonisch unsauber.

4.5 Hardware-Grenzen und Ambitionen

Ich hatte die Ambition, riesige lokale KI-Modelle (wie Qwen3 235b) zu nutzen, um vollständig unabhängig von Firmen wie OpenAI zu sein. Ich dachte, meine Hardware zuhause würde reichen. Doch ich unterschätzte den Unterschied zwischen "laufen lassen" und "benutzbar sein", geschweige denn «skalierbar sein». Ein 235-Milliarden-Parameter-Modell auf einer CPU laufen zu lassen, weil sich eine Grafikkarte mit ausreichendem Speicher schon im Bereich von mehreren Tausend Franken befindet, bedeutet, dass etwa ein Satz pro Minute generiert wird, und das ist auf zwei meiner Intel Xeon e2690-v3 CPUs mit insgesamt 24 Kernen und 256gb DDR4-RAM. Das ist für einen interaktiven Chatbot unbrauchbar. Ich musste lernen, meine Ambitionen an die Realität der verfügbaren Hardware anzupassen und auf effizientere Modelle (DeepSeek via API oder kleinere Gemma-Modelle) umzusteigen.

5. Resultate und Beantwortung der Fragestellung

5.1 Erreichung der Minimalziele

Vergleicht man das Endprodukt mit den Zielen aus dem ursprünglichen Konzeptpapier, so wurden die Minimalziele vollständig und erfolgreich erreicht.

● Die Dokumentenverarbeitung funktioniert zuverlässig. Uploads werden verarbeitet, Text wird erkannt (auch dank Tesseract bei Bilddateien) und korrekt in ChromaDB gespeichert.

● Die RAG-Engine liefert kontextbezogene Antworten. In diversen Tests konnte gezeigt werden, dass der Bot spezifische Fragen zu alten ZAP-Prüfungen (z.B. "Lösungsweg Aufgabe 3, ZAP 2022") korrekt beantwortet, die ein Standard-ChatGPT mangels Wissens nicht lösen könnte.

● Das Chat-Interface ist relativ stabil, unterstützt Streaming und ermöglicht eine flüssige Interaktion, die sich für den Nutzer natürlich anfühlt.

5.2 Erreichung der Maximalziele

Bei den Maximalzielen zeigt sich ein differenziertes Bild, das die Grenzen eines Ein-Personen-Projekts aufzeigt.

● Erreicht: Das UI ist einigermassen professionell geworden (Dank viel AI-CSS-Arbeit und MathJax-Integration). Features wie die Prüfungssimulation und die automatische Aufgabengenerierung ("Erstelle mir 5 Algebra-Aufgaben auf Niveau Langgymnasium") funktionieren und bieten einen echten Mehrwert.

● Teilweise erreicht: Die "Hochleistungs-RAG-Engine" ist eher eine solide Basis-Version. Hybrides Retrieval ist rudimentär implementiert, jedoch laut meinen Tests mit einer hohen Erfolgsquote, aber komplexe Re-Ranking-Algorithmen (die Ergebnisse nochmals sortieren für Optimale Suchresultate) fehlen noch.

● Verfehlt: Die Unterstützung für handschriftliche Notizen ist kaum nutzbar. Die Open-Source-OCR (Tesseract) scheitert oft an Handschrift. Hier wären teure Cloud-Vision-APIs (Google/Azure) nötig gewesen, was den finanziellen Rahmen gesprengt hätte.

Die App läuft, ist aber noch stark an die spezifische Server-Umgebung gebunden.

5.3 Funktionsweise des Endprodukts

"Gymiboost" präsentiert sich heute als funktionales Dashboard. Der Schüler loggt sich ein, wählt ein Fach (Deutsch/Mathe) oder lädt eigene Unterlagen hoch. Er kann dann entweder frei chatten ("Erkläre mir diese Aufgabe"), wobei die KI im Hintergrund die passenden Lösungen aus der Datenbank zieht, oder er startet den "Aufgabengenerator". Letzterer ist das technische Highlight: Der Schüler wählt ein Thema und einen Schwierigkeitsgrad (1-5), und die KI generiert ein massgeschneidertes Aufgabenblatt inklusive separatem Lösungsblatt. Ausserdem kann er sich eine Prüfung mit echtem Zeitdruck simulieren und diese von der KI korrigieren und bewerten lassen, oder einen KI-gefertigten Kurs zu einem ihm spezifischen Thema generieren und Absolvieren. Das System nutzt hierfür das GPT-5 Ökosystem via API, da es besonders gut strukturierten Output und Agentische Leistungen liefern kann.

5.4 Antwort auf die Leitfrage

Die eingangs gestellte Frage lautete: "Kann ich als einzelner Entwickler eine RAG-basierte Web-Applikation konzipieren und implementieren, die eine halluzinationsfreie, prüfungsspezifische Unterstützung für die ZAP (Langgymnasium) bietet, und lassen sich die dabei auftretenden technischen sowie architektonischen Hürden mit begrenzten Ressourcen überwinden?"

Die Antwort lautet: Ja, aber mit klaren Einschränkungen.

Es ist mir gelungen, eine Applikation zu entwickeln, die spezifisches ZAP-Wissen bereitstellt und das Problem der Halluzinationen durch RAG massiv reduziert. Die technischen Hürden (Datenbanken, Encoding, MathJax) liessen sich durch Beharrlichkeit überwinden, erforderten jedoch einen unverhältnismässig hohen Zeitaufwand für Fehlersuche und Debugging (geschätzt auf etwa 400 Stunden insgesamt). Die Einschränkung liegt in der "Ein-Person"-Natur des Projekts: Gewisse Maximalziele wie perfekte Handschrifterkennung, eine perfekte User experience oder Cloud-Skalierung erfordern Ressourcen, die im Rahmen einer Maturarbeit kaum aufzubringen sind. Das Projekt ist ein erfolgreicher "Proof of Concept", der funktioniert, aber für einen kommerziellen, landesweiten Einsatz noch sehr viel Arbeit und ein Team benötigen würde.

6. Fazit und Reflexion

6.1 Zusammenfassung

Diese Arbeit zeigt exemplarisch, dass es heute möglich ist, mit begrenzten Mitteln und viel Eigeninitiative eine Software zu entwickeln, die in ihrer Komplexität vor wenigen Jahren noch Forschungslaboren vorbehalten war. Durch die intelligente Kombination von flexiblen Frameworks (Flask), lokalen Vektordatenbanken (ChromaDB) und mächtigen KI-Modellen (DeepSeek/Gemma) ist "Gymiboost" entstanden: Ein Werkzeug, das nicht nur Antworten gibt, sondern den Lernprozess aktiv unterstützt und personalisiert.

6.2 Persönliche Reflexion

Rückblickend war dieses Projekt weniger eine Übung im Programmieren als eine Lektion in Demut und Problemlösung. Ich habe gelernt, dass 80 % der Zeit nicht in das Schreiben von neuem Code fliessen, sondern in das Beheben von Fehlern, die man sich selbst durch falsche Annahmen eingebrockt hat. Ich habe mich oft gequält. Die Komplexität der Systeme und Frameworks kann oft erdrückend sein. Es wäre einfacher gewesen ("Ignorance is bliss"), eine fertige Baukasten-Lösung zu nehmen. Aber dann hätte ich nicht verstanden, warum Dinge funktionieren oder scheitern, was ja der Punkt der Arbeit war. Die Entscheidung für Flask war goldrichtig. Sie zwang mich, die Mechanismen im Detail zu verstehen. Hätte ich Django genutzt, hätte ich vielleicht schneller ein Ergebnis gehabt, aber ich hätte weniger gelernt.

Am Ende steht die Erkenntnis: Intelligenz allein ist noch keine Kompetenz, aber die Fähigkeit, seine Intelligenz durch selbstgebaute Werkzeuge wirksam zu machen, gibt ein tiefes Gefühl von Selbstwirksamkeit. Und das ist für die eigene Bildung vielleicht wertvoller als jede bestandene Prüfung.

7. Quellenverzeichnis

Aufgrund der Chaotischen Natur des Programmierens und des Debuggings, wurde mir von meinem Coach geraten, meine nutzung von KI und Web-foren eher Gebündelt anzugeben, als bei jeder Linie code (insgesamt über 10`000), anzugeben, wie genau sie entstanden ist. Daher erkläre ich hiermit, dass Teile des Codes, wie oben schon genannt, hauptsächlich im Frontend aber auch einiges im Backend, mithilfe von KI geschrieben, revidiert, formatiert oder debugged wurden, sowie ich auch oft mich auf Online-Foren wie Reddit und Stackoverflow im Rahmen dieser Arbeit sowie aus Eigeninteresse weiterbildete bzw. mich mit Kameraden Lösung Architektonischer und Struktureller Probleme in meinem Code konsultierte. Unten befinden sich jedoch einige der Quellen, welche Essenziell für meine Design-entscheidungen sowie Problemlösung waren.

die meisten der Quellen wurden am 4. Januar erneut aufgesucht und konsultiert, um dieses Quellenverzeichnis zu gestalten:

1. Django vs Flask : r/learnpython - Reddit, Zugriff am Januar 4, 2026,

https://www.reddit.com/r/learnpython/comments/1bfmx3x/django_vs_flask/

2. Which Is the Best Python Web Framework: Django, Flask, or FastAPI? | The

PyCharm Blog, Zugriff am Juli, 2025,

https://blog.jetbrains.com/pycharm/2025/02/django-flask-fastapi/

3. Flask or Django: Which One Best Fits Your Python Project? | AppSignal Blog, Zugriff

am Juni, 2025,

https://blog.appsignal.com/2025/06/25/flask-or-django-which-best-fits-your-python-pr

oject.html

4. I feel like Flask is so much better than Django - Reddit, Zugriff am Januar 4, 2026,

https://www.reddit.com/r/flask/comments/vtwysh/i_feel_like_flask_is_so_much_better_than_django/

5. FastAPI vs Flask: The Ultimate Comparison - ClickIT, Zugriff am Januar 4, 2026,

https://www.clickittech.com/software-development/fastapi-vs-flask/

6. Flask vs FastAPI?: r/Python - Reddit, Zugriff am Januar 4, 2026,

https://www.reddit.com/r/Python/comments/ujoggf/flask_vs_fastapi/

7. Chroma vs. Pinecone: Which Vector Database Should You Use? | by Tahir | Medium,

Zugriff am Januar 4, 2026,

https://medium.com/@tahirbalarabe2/chroma-vs-pinecone-which-vector-database-should-you-use-e7dcfb1a9aa3

8. Pinecone vs Chroma 2025: Complete Vector Database Comparison | Performance, Cost, Scale - Aloa, Zugriff am Januar 4, 2026, https://aloa.co/ai/comparisons/vector-database-comparison/pinecone-vs-chroma

10. Release notes | Gemini API - Google AI for Developers, Zugriff am Januar 4, 2026, https://ai.google.dev/gemini-api/docs/changelog

11. google/gemma-3-27b-it - Hugging Face, Zugriff am Juli , 2025, https://huggingface.co/google/gemma-3-27b-it

14. DeepSeek R1 vs GPT-4o: Which AI Model Excels in RAG? | by Akash A Desai | Medium, Zugriff am Januar 4, 2026, https://aksdesai1998.medium.com/deepseek-r1-vs-gpt-4o-which-ai-model-excels-in-rag-5b3b3f78f817

15. MySQL: Large VARCHAR vs. TEXT? - Stack Overflow, Zugriff am Januar 4, 2026, https://stackoverflow.com/questions/2023481/mysql-large-varchar-vs-text

16. Maximum length for MySQL type text - database - Stack Overflow, Zugriff am Januar 4, 2026, https://stackoverflow.com/questions/6766781/maximum-length-for-mysql-type-text

17. Allow MySQL to store emojis in utf8mb4 encoding? - Stack Overflow, Zugriff am Juli, 2025, https://stackoverflow.com/questions/38698959/allow-mysql-to-store-emojis-in-utf8mb4-encoding

18. Problems with inserting emojis into database using mysql.connector in python, Zugriff am Juli, 2025, https://stackoverflow.com/questions/66879706/problems-with-inserting-emojis-into-data

base-using-mysql-connector-in-python

19. Web Apps for work projects: Did you choose Flask or Django? And why? Any learnings? : r/Python - Reddit, Zugriff am Juni, 2025,

https://www.reddit.com/r/Python/comments/16nfteh/web_apps_for_work_projects_did_you_choose_flask/

20. Newest 'rag' Questions - Stack Overflow, Zugriff am Januar 4, 2026,

https://stackoverflow.com/questions/tagged/rag?tab=Newest

21. Integrating Stripe with Flask: A Step-by-Step Tutorial : r/flask - Reddit, Zugriff am

Juli, 2025,

https://www.reddit.com/r/flask/comments/1fdzhr7/integrating_stripe_with_flask_a_stepbystep/

8. Eigenständigkeitserklärung

Ich, Luka Simonovic erkläre hiermit, dass ich die vorliegende Maturitätsarbeit eigenständig und ohne unerlaubte fremde Hilfe erstellt habe und dass alle Quellen, Hilfsmittel und Internetseiten wahrheitsgetreu verwendet wurden und belegt sind. Zudem habe ich für die Erstellung der Maturitätsarbeit die KI nicht in unerlaubter Weise verwendet.

Ich bin nicht damit einverstanden, dass eine Kopie meiner Maturitätsarbeit bei einer Anfrage nach aussen abgegeben wird.

Unterschrift: _____ Bülach, 4. Januar 2025